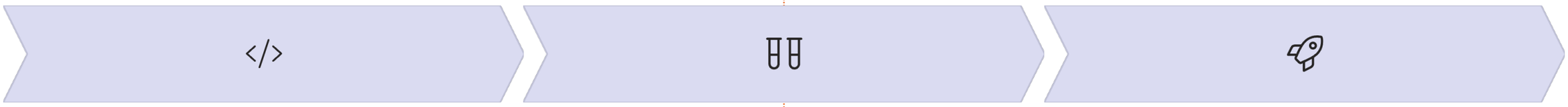
Persistent

- **Building Your First Pipeline**

by Vishwanath MS

Building Your First Pipeline: Overview

A CI/CD pipeline is an automated sequence of steps in software development, from code changes to deployment. It streamlines the delivery process, ensuring efficient, rapid, and reliable software releases.



Development

Developers write and commit code to a version control system.

Testing

Automated tests run to verify functionality, performance, and security.

Deployment

Validated code is automatically released to production environments.

The entire pipeline is defined in a configuration file, typically

```
.gitlab-ci.yml
```

for GitLab CI/CD.

Persistent

Anatomy of .gitlab-ci.yml

The entire CI/CD pipeline's structure and behavior are defined in a single `.gitlab-ci.yml` file, located at the project root.

YAML Format

A human-readable data serialization language that controls the entire pipeline.

Stages

Defines the sequential phases of the pipeline, like **build**, **test**, and **deploy**.

Jobs

Specific tasks that belong to stages and define what commands to run.

Scripts

The actual commands executed by each job, specified within the job's definition.

Defining Your Pipeline: Stages, Jobs, Scripts

The structure of your pipeline is defined by three key sections:

- Stages: Define the sequential phases of your pipeline (e.g., build, test, deploy).
- Jobs: Named sections representing specific tasks executed within a chosen stage.
- Script: Contains the actual commands that a job runs.

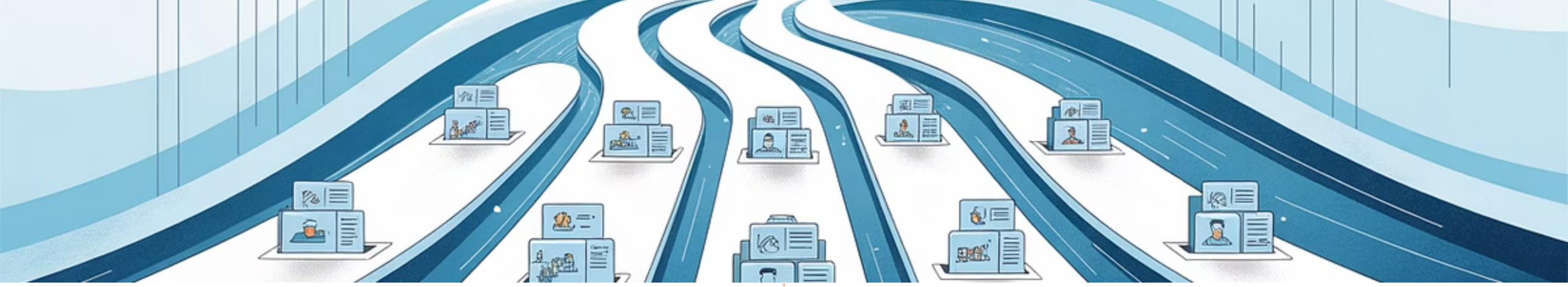
Here's an example showing how these components work together:

```
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - echo "Building the application"

test-job:
  stage: test
  script:
    - echo "Running tests"

deploy-job:
  stage: deploy
  script:
    - echo "Deploying application"
```



Defining Stages – Best Practices

Understanding how stages operate is crucial for designing efficient and reliable CI/CD pipelines. Here are key principles:

Common Stages

Leverage standard stage names like **build**, **test**, **deploy**, **lint**, and **review** for clarity and consistency across projects.

Sequential Flow

Stages execute in a defined order, from top to bottom as listed in your `.gitlab-ci.yml`, ensuring logical progression.

Parallel Jobs

Jobs within the same stage run concurrently, significantly speeding up pipeline execution and maximizing resource utilization.

Job Configuration Options

Beyond the basic stages and scripts, GitLab CI/CD jobs offer powerful configuration options to control their behavior:

Script

Defines the exact commands executed by the job. This is the operational core of your job.

Only / Except

Controls when a job runs based on conditions like branches, tags, or specific file changes.

Tags

Assigns jobs to specific GitLab Runners with matching tags, ensuring they run in appropriate environments.

Artifacts

Specifies files or directories produced by a job that should be saved, shared, or downloaded for later use.

Here’s an example showing these options in action:

```
test-job:
  stage: test
  tags:
    - python
  script:
    - npm install
    - npm test
  artifacts:
    paths:
      - coverage/
```

Hands-On: Building Your First Pipeline

Let's put theory into practice by creating a minimal CI/CD pipeline. Our goal is to set up a basic workflow that:

Builds the Application

Ensuring your code compiles successfully.

Runs Tests

Verifying the application's functionality.

Reports Status

Providing immediate feedback on pipeline execution.

Begin by creating a new file named `.gitlab-ci.yml` at the root of your project repository and add the following content:

```
stages:
  - build
  - test

build-job:
  stage: build
  script:
    - echo "Compiling code..."

test-job:
  stage: test
  script:
    - echo "Running unit tests..."
```

Once the file is created, **add, commit, and push** it to your repository. GitLab CI will automatically detect this file and initiate your first pipeline!

Persistent